

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

Screenshot:

```
remote: Enumerating objects: 198, done.
remote: Counting objects: 100% (198/198), done.
remote: Compressing objects: 100% (140/140), done.
remote: Total 198 (delta 25), reused 197 (delta 24), pack-reused 0 (from 0)
Receiving objects: 100% (198/198), 2.94 MiB | 115.00 KiB/s, done.
Resolving deltas: 100% (25/25), done.

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A
$ cd IT314AB_Moreno-A

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A/IT314AB_Moreno-A (main)
$ git checkout features/widgets
branch 'features/widgets' set up to track 'origin/features/widgets'.
Switched to a new branch 'features/widgets'

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A/IT314AB_Moreno-A (features/widgets)
$ git checkout -b feature/data-ui
Switched to a new branch 'feature/data-ui'

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A/IT314AB_Moreno-A (feature/data-ui)
$ git branch
* feature/data-ui
  features/widgets
  main

User@DESKTOP-GL91V6M MINGW64 ~/Documents/Moreno_A/IT314AB_Moreno-A (feature/data-ui)
$ |
```

Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Create:

- Your identified data
- The corresponding data types

name	string
age	int
height	Double
Status	Bool

Flag 2 — Create Your Data

Screenshot:

```
String name = "Anthony";
String myCourseSection = "BSIT";
int age = 21;
String hobby = "gaming";
double height = 5.7;
bool isStudent = true;

void main() {
```

Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

Test:

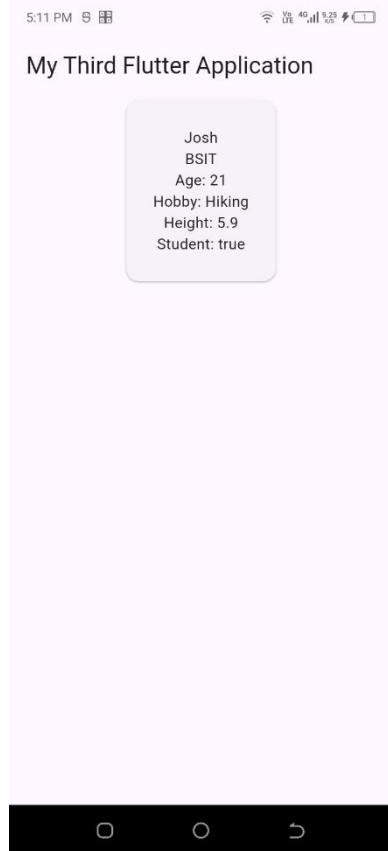
Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

```
crossAxisAlignment: CrossAxisAlignment.center,
children: [
  Padding(
    padding: const EdgeInsets.all(25.0),
    child: Column(
      children: [
        Text(name),
        Text(myCourseSection),
        Text('Age: $age'),
        Text('Hobby: $hobby'),
        Text('Height: $height'),
        Text('Student: $isStudent'),
```

```
1 import 'package:flutter/material.dart';
2
3 String name = "Josh";
4 String myCourseSection = "BSIT";
5 int age = 21;
6 String hobby = "Hiking";
7 double height = 5.9;
8 bool isStudent = true;
9
10 void main() {
```



Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

Use the variable when displaying the image.

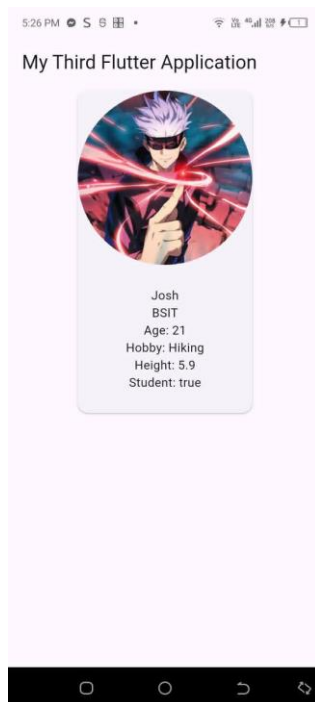
Then change the variable to another registered image.

The image displayed by the application should change.

Screenshot:

```
Dart SDK is not configured Download Dart SDK Op
1 import 'package:flutter/material.dart';
2
3 String name = "Josh";
4 String myCourseSection = "BSIT";
5 int age = 21;
6 String hobby = "Hiking";
7 double height = 5.9;
8 bool isStudent = true;
9 String imagePath = 'assets/images.jpg';
10
11 void main() {
12   runApp(const MyApp());
13 }
14
```

```
Card(
1   child: Column(
2     mainAxisAlignment: MainAxisAlignment.center,
3     crossAxisAlignment: CrossAxisAlignment.center,
4     children: [
5       CircleAvatar(
6         radius: 100,|
7         backgroundImage: AssetImage(imagePath),
8       ),
9     ],
10  )
11 )
```



🚩 Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

```
List<Profile> profiles = [
    Profile(
        name: "Gojo",
        courseSection: "BSIT",
        age: 21,
        hobby: "Hiking",
        height: 5.9,
        isStudent: true,
        imagePath: "assets/images.jpg",
        color: Colors.purple,
    ),
    Profile(
        name: "Sukuna",
        courseSection: "CURSE",
        age: 1000,
        hobby: "AURA",
        height: 6.2,
        isStudent: false,
        imagePath: "assets/sukuna.avif",
        color: Colors.red,
    ),
    Profile(
        name: "Toji",
        courseSection: "Jobless",
        age: 30,
        hobby: "Gambler",
        height: 6.2,
        isStudent: false,
        imagePath: "assets/Toji1.jfif",
        color: Colors.green,
    ),
    Profile(
        name: "Yuji",
        courseSection: "Sorcerer",
        age: 18,
        hobby: "Running",
        height: 5.8,
        isStudent: true,
        imagePath: "assets/yuji.jfif",
        color: Colors.blue,
    ),
]
```

```
Profile(  
    name: "Yuta",  
    courseSection: "High Sorcerer",  
    age: 22,  
    hobby: "Travel",  
    height: 5.9,  
    isStudent: true,  
    imagePath: "assets/yuta.jfif",  
    color: Colors.orange,  
),  
];  
  
void main() {
```

Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

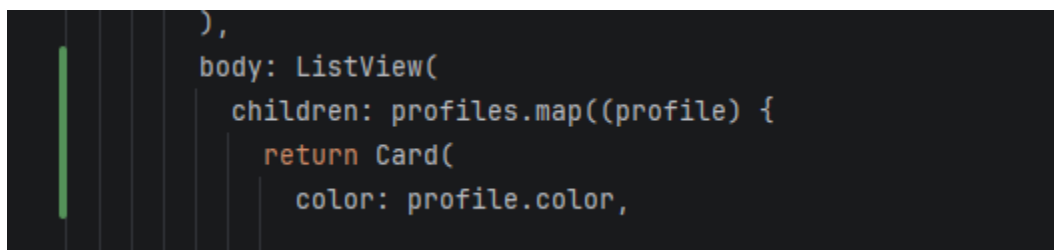
Requirement:

The user must be able to:

Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

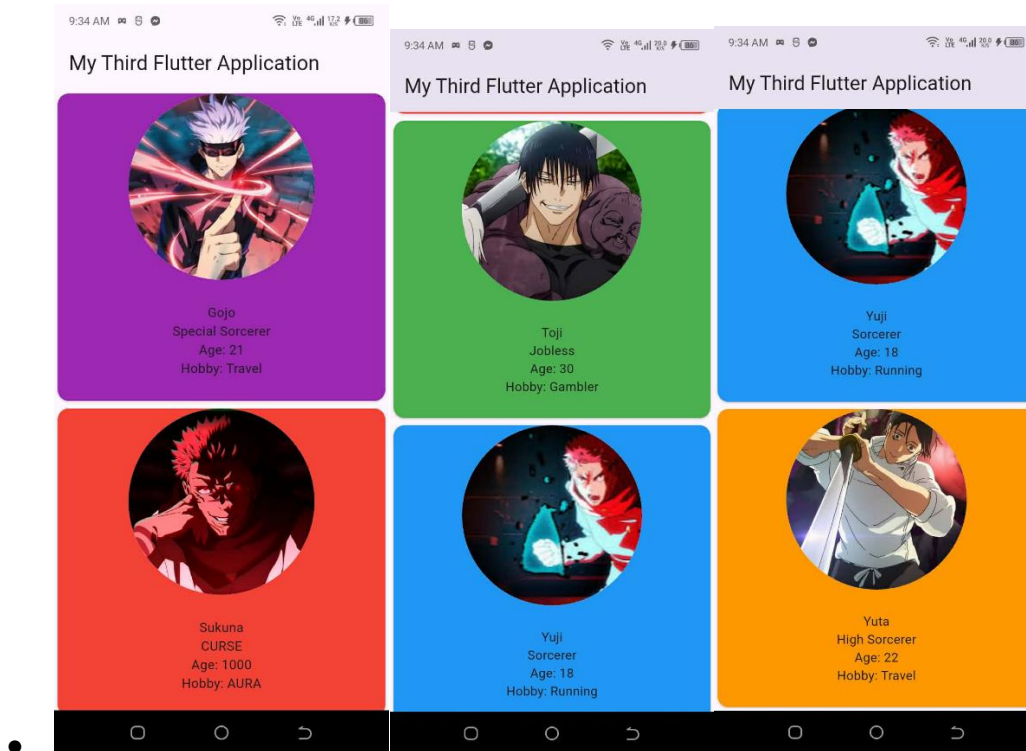
Screenshot:

A screenshot of a code editor showing a Flutter widget definition. The code is as follows:

```
),  
body: ListView(  
  children: profiles.map((profile) {  
    return Card(  
      color: profile.color,
```

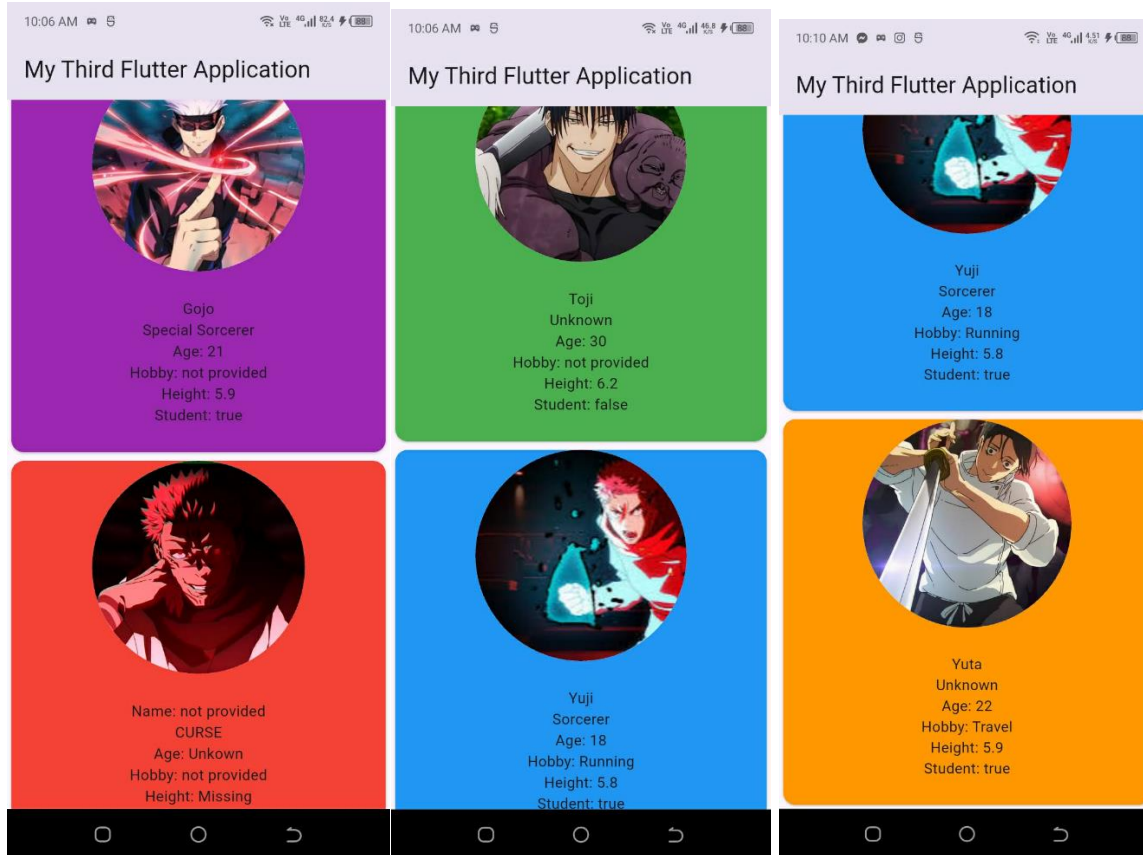
The code is written in a dark-themed editor with a light green vertical cursor on the left side.

To list & customscrollview are just same so I decided to use listview



🚩 Flag 7 — Survive the Database

Screenshot:



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

```
C:\Users\User\Documents\Moreno_A\IT314AB_Moreno-A>git commit -m "feat: adding project flag 3"
[feature/data-ui 64ade51] feat: adding project flag 3
12 files changed, 2523 insertions(+), 122 deletions(-)
create mode 100644 .idea/IT314AB_Moreno-A.iml
create mode 100644 .idea/deviceManager.xml
create mode 100644 .idea/misc.xml
create mode 100644 .idea/modules.xml
create mode 100644 .idea/vcs.xml
create mode 100644 .idea/workspace.xml
create mode 100644 my_first_flutter_app/assets/Toji1.jfif
create mode 100644 my_first_flutter_app/assets/sukuna.avif
create mode 100644 my_first_flutter_app/assets/yuji.jfif
create mode 100644 my_first_flutter_app/assets/yuta.jfif
C:\Users\User\Documents\Moreno_A\IT314AB_Moreno-A>
```